

Cross Hypervisor DR UI And Backend Flow Diagrams

작성일: 2026-07-01

대상 브랜치: `feature/ftctl-cloud-integration`

상위 문서:

- [500-cross-hypervisor-dr-architecture-plan-20260630.md](#)
- [506-cross-hypervisor-dr-cloud-ui-design-20260630.md](#)
- [507-cross-hypervisor-dr-cloud-api-command-design-20260630.md](#)
- [508-cross-hypervisor-dr-cloud-backend-wiring-design-20260630.md](#)
- [511-cross-hypervisor-dr-implementation-progress-20260630.md](#)
- [513-cross-hypervisor-dr-integration-smoke-and-deployment-prep-20260701.md](#)

1. 목적

이 문서는 배포 전 리뷰를 위해 Cross Hypervisor DR의 UI 흐름과 backend 흐름을 한 곳에서 설명한다.

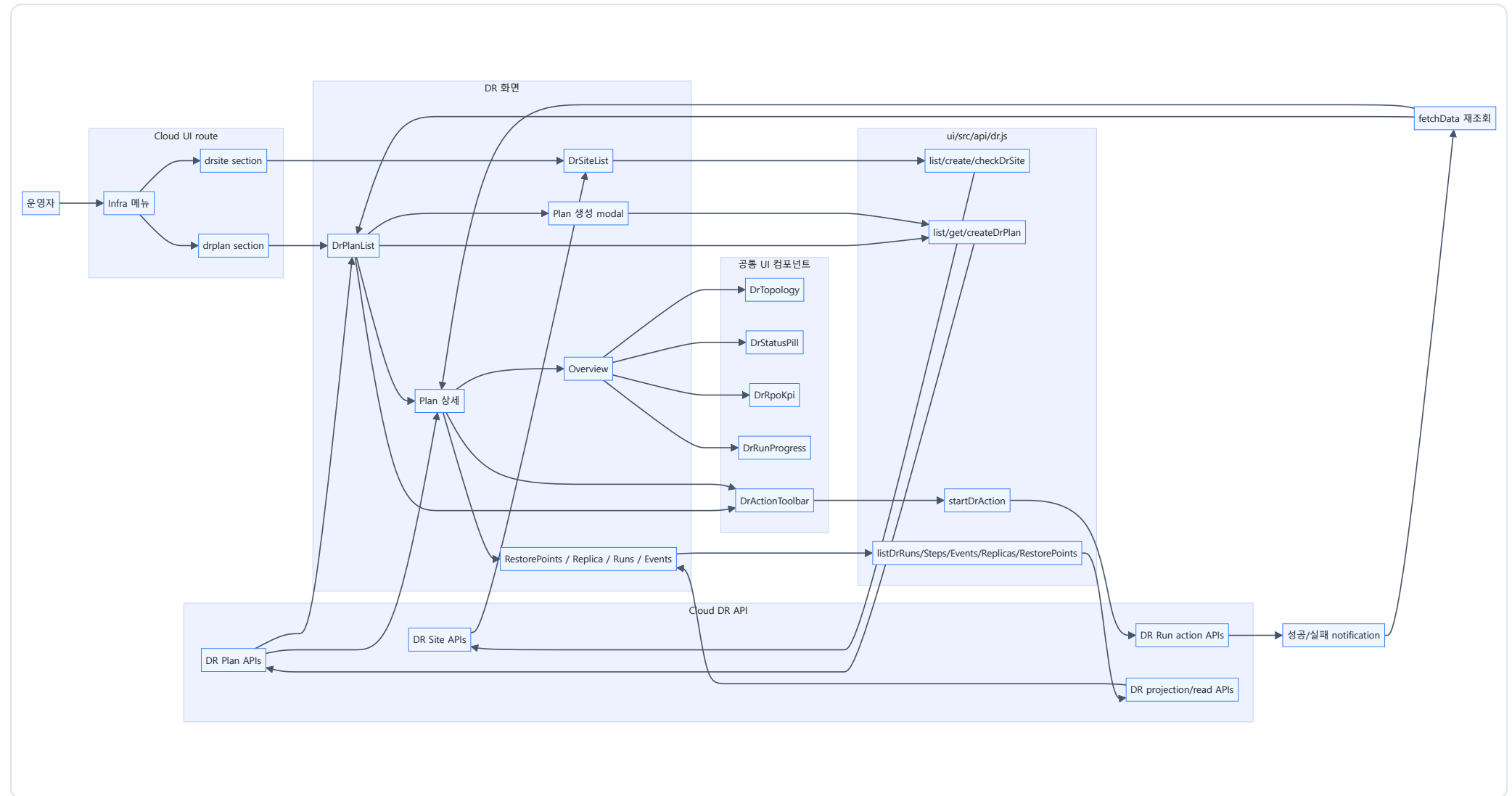
다이어그램은 A4 출력 기준으로 작성했다.

- UI 흐름 다이어그램: A4 가로 1장
- Backend 흐름 다이어그램: A4 가로 1장
- UI-to-backend 실행 시퀀스: A4 가로 1장

Markdown 렌더러에서 PDF로 출력할 때는 `A4`, `Landscape`, margin `10-12mm` 를 기준으로 한다. GitHub Markdown처럼 print CSS가 무시되는 렌더러에서는 Mermaid 다이어그램을 한 장씩 별도 출력한다.

2. UI 관점 흐름

DIAGRAM 1



2.1 UI 흐름 설명

1. 운영자는 Infra 메뉴에서 `drsite` 또는 `drplan` section으로 진입한다.
2. `drsite` section은 `listDrSites` 권한이 있을 때 `DrSiteList.vue` 를 로드한다.
3. `drplan` section은 `listDrPlans` 권한이 있을 때 `DrPlanList.vue` 를 로드한다.
4. `DrPlanList.vue` 의 목록 화면은 `listDrSites`, `listDrPlans` 를 함께 호출해 site 이름과 plan 목록을 조합한다.
5. Plan 생성 modal은 `createDrPlan` 을 호출한 뒤 목록을 다시 조회한다.
6. Plan 상세 화면은 `getDrPlan`, `listDrRuns` 를 호출하고, 선택 탭에 따라 restore point, replica, run step, event API를 추가 호출한다.
7. `DrActionToolbar.vue` 는 plan response의 `actioneligibility` 를 기준으로 action 버튼을 활성화한다.
8. action 버튼은 UI에서 host, qemu, FTCTL runtime을 직접 호출하지 않고 `ui/src/api/dr.js` 의 `startDrAction` 을 통해 Cloud API만 호출한다.
9. action API 응답으로 `DrRun` 이 반환되면 notification을 표시하고 `fetchData` 로 plan/run 상태를 다시 읽는다.
10. 진행 상태는 `DrRunProgress.vue`, 상태는 `DrStatusPill.vue`, RPO는 `DrRpoKpi.vue`, site 관계는 `DrTopology.vue` 가 표시한다.

2.2 UI 구현 기준 파일

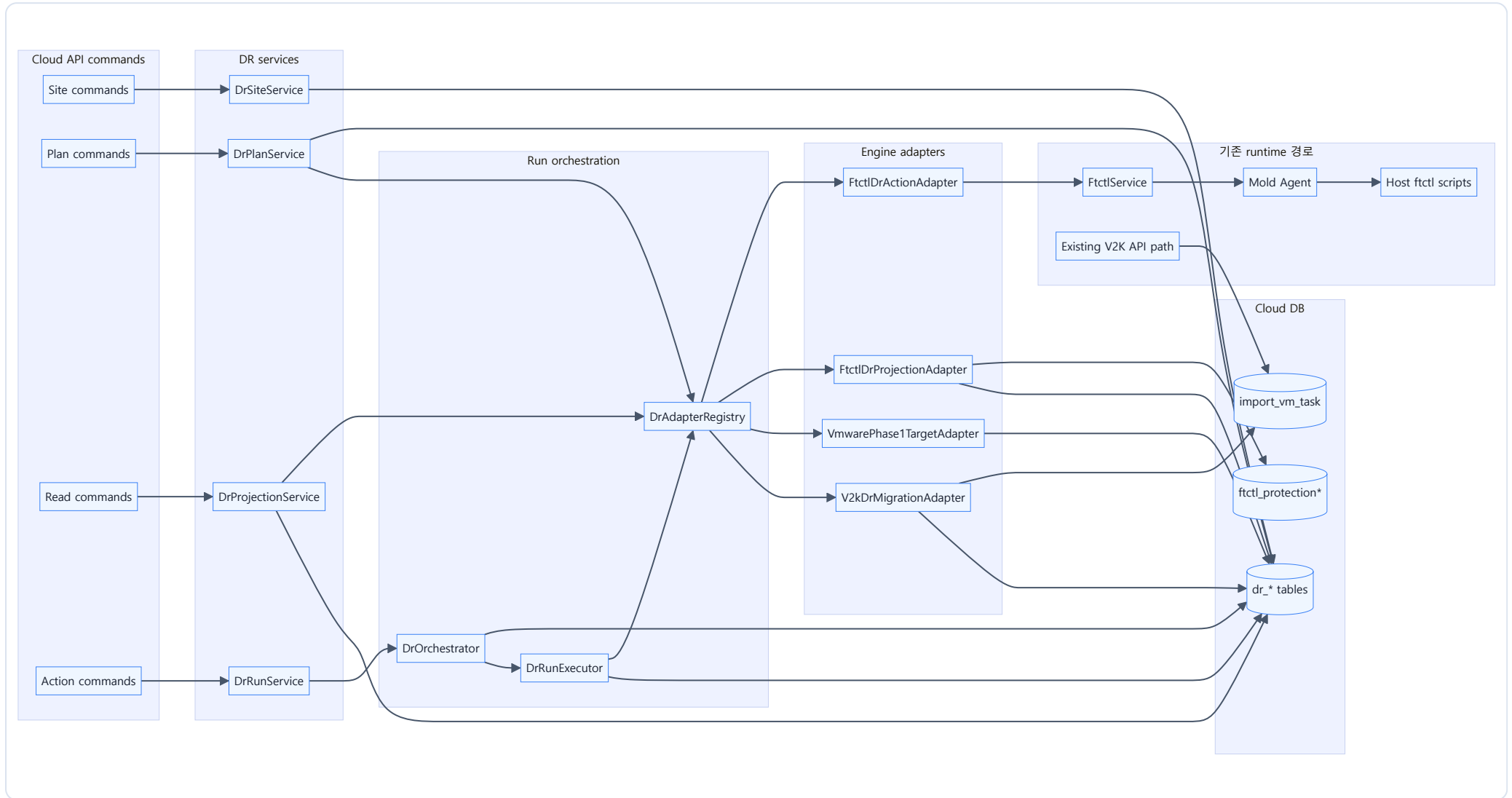
영역	파일	역할
메뉴/route	<code>ui/src/config/section/infra/drSite.js</code>	DR Sites section 등록
메뉴/route	<code>ui/src/config/section/infra/drPlan.js</code>	DR Plans section 등록
API wrapper	<code>ui/src/api/dr.js</code>	DR API 호출, list/object response normalize
Plan 화면	<code>ui/src/views/infra/dr/DrPlanList.vue</code>	plan 목록, 상세, 생성, action 실행
Overview	<code>ui/src/views/infra/dr/DrPlanOverview.vue</code>	topology, KPI, current run 표시
Action	<code>ui/src/components/dr/DrActionToolbar.vue</code>	action eligibility 기반 버튼 gating
Progress	<code>ui/src/components/dr/DrRunProgress.vue</code>	run 진행률 표시
Style	<code>ui/src/style/cross-dr.less</code>	Cross DR 공통 스타일, 다크모드 변수

2.3 UI 배포 전 확인 포인트

- active bundle에 `fetchDrPlans`, `DrRunProgress`, `cross-dr`, `listDrSites`, `startDrSync` marker가 있어야 한다.
- `ko_KR.json`, `en.json` 에 DR label/message가 포함되어야 한다.
- `/usr/share/cloudstack-management/webapp/WEB-INF` 를 보존하고 정적 asset만 갱신해야 한다.
- `/client/` 가 HTTP 200을 반환해야 한다.
- 버튼 활성화는 UI 단독 판단이 아니라 backend가 응답한 `actioneligibility` 를 기준으로 한다.

3. Backend 관점 흐름

DIAGRAM 2



3.1 Backend 흐름 설명

1. Cloud API command는 `DisasterRecoveryClusterServiceImpl.getCommands()` 에 등록된다.
2. 목록/조회/생성 API는 `DrSiteService`, `DrPlanService`, `DrProjectionService` 를 호출한다.
3. action API는 `AbstractDrPlanActionCmd` 계열 command가 공통으로 처리하고 `DrRunService.startRun` 을 호출한다.
4. `DrRunServiceImpl` 은 `DrOrchestrator.createRun` 으로 `dr_run`, 초기 `dr_run_step`, `dr_event` 를 만든다.
5. `DrOrchestrator.executeRun` 은 현재 구현에서 즉시 `DrRunExecutor.queueRun` 을 호출한다.
6. `DrRunExecutorImpl` 은 run을 `RUNNING` 으로 바꾸고 `execute` step과 event를 기록한 뒤 adapter를 선택한다.
7. adapter 선택 key는 `engineType:engineBindingType` 이다.
8. FTCTL plan은 `FTCTL:FTCTL` key로 `FtctlDrActionAdapter` 와 `FtctlDrProjectionAdapter` 를 사용한다.
9. VMware Phase 1 plan은 `VMWARE_PHASE1:VMWARE_PHASE1` key로 target skeleton/readiness record를 만든다.
10. V2K plan은 `V2K:V2K` key로 기존 `import_vm_task` 의 phase1/phase2 상태를 추적한다.
11. adapter 성공 시 run step/event를 성공으로 기록하고 `DrProjectionService.refreshPlanProjection` 을 best-effort로 수행한다.
12. adapter 실패 시 run step/event와 run error code/message를 기록하고 UI가 이를 `DrRunProgress`, Events tab에서 읽는다.

3.2 Engine별 책임 경계

Engine	Direction	Backend adapter	책임	명시적 제외
FTCTL	KVM_TO_KVM	<code>FtctlDrActionAdapter</code> , <code>FtctlDrProjectionAdapter</code>	기존 FTCTL 보호/복구 성공 경로 위임, FTCTL 상태 projection	host/qemu script 재구현
VMware Phase 1	KVM_TO_VMWARE	<code>VmwarePhase1TargetAdapter</code>	VMware target skeleton/readiness record 생성	vCenter VM 실제 생성/cutover
V2K	VMWARE_TO_KVM	<code>V2kDrMigrationAdapter</code>	기존 import VM task phase1/phase2 상태 추적, DR run step 기록	phase2 시작 API 직접 호출

3.3 Backend 구현 기준 파일

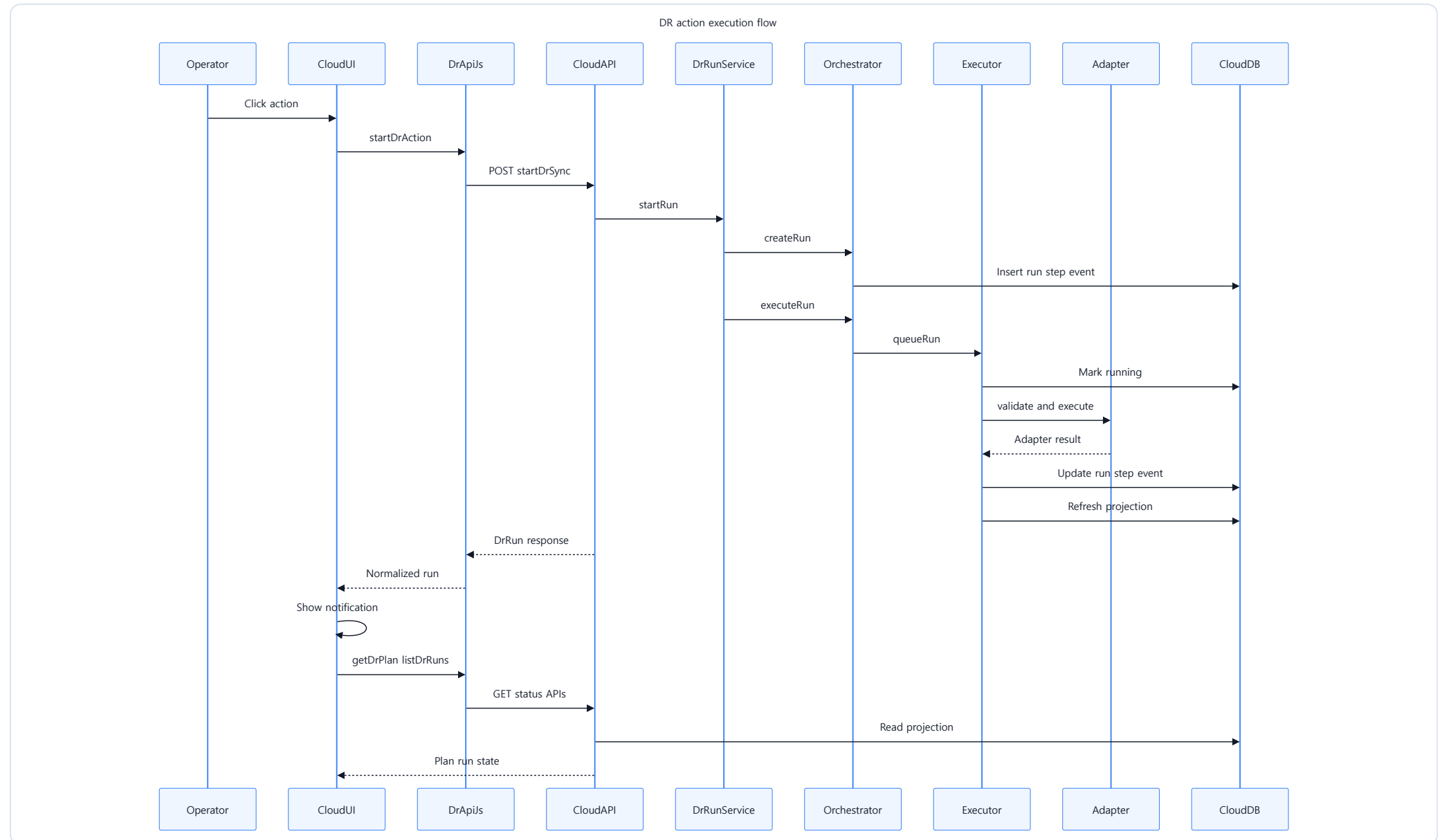
영역	파일	역할
API 등록	DisasterRecoveryClusterServiceImpl.java	DR API command 등록
Action command	AbstractDrPlanActionCmd.java	action request를 DrRun 으로 변환
Plan service	DrPlanServiceImpl.java	plan CRUD, action eligibility
Run service	DrRunServiceImpl.java	run 시작/조회/cancel
Orchestrator	DrOrchestratorImpl.java	run 생성, step/event 초기화
Executor	DrRunExecutorImpl.java	adapter 실행, 성공/실패 기록
Registry	DrAdapterRegistryImpl.java	engine key 기반 adapter 선택
FTCTL	FtctlDrActionAdapter.java	기존 FtctlService action 위임
FTCTL projection	FtctlDrProjectionAdapter.java	ftctl_protection* 상태를 dr_* 로 투영
VMware Phase 1	VmwarePhase1TargetAdapter.java	skeleton replica record 생성
V2K	V2kDrMigrationAdapter.java	import_vm_task phase 상태 추적
Spring	spring-disaster-recovery-context.xml	DAO/service/orchestrator/adapter bean 등록

3.4 Backend 배포 전 확인 포인트

- cloud-plugin-integrations-disaster-recovery jar에 다음 class/resource가 포함되어야 한다.
 - DrPlanServiceImpl.class
 - DrRunExecutorImpl.class
 - FtctlDrActionAdapter.class
 - FtctlDrProjectionAdapter.class
 - VmwarePhase1TargetAdapter.class
 - V2kDrMigrationAdapter.class
 - spring-disaster-recovery-context.xml
- DB에는 dr_site, dr_plan, dr_replica, dr_run, dr_run_step, dr_event 등 신규 dr_* table이 있어야 한다.
- management log에 Spring bean load 실패, API command registration 충돌, schema 누락 오류가 없어야 한다.
- 기존 FTCTL KVM-to-KVM 성공 경로는 FtctlService 를 통해 호출되므로 qemu/ftctl package 변경 없이 회귀 확인한다.

4. UI-to-backend 실행 시퀀스

DIAGRAM 3



4.1 실행 시퀀스 설명

1. UI action 버튼은 `DrActionToolbar` 에서 `actioneligibility` 가 true인 경우에만 실행된다.
2. UI는 `startDrAction(command, { planid })` 만 호출한다.
3. Cloud API command는 run type을 `SYNC`, `TEST_FAILOVER`, `FAILOVER`, `FAILBACK`, `REPROTECT`, `ADOPT` 등으로 매핑한다.
4. `DrRunService` 는 run을 만들고 현재 구현에서는 즉시 `executor`를 호출한다.
5. `executor`는 adapter 실행 전후로 `dr_run_step`, `dr_event`, `dr_run` 을 갱신한다.
6. adapter 결과가 성공이면 projection refresh를 수행하고, 실패이면 error code/message를 run과 step에 남긴다.
7. UI는 최초 응답의 `DrRun` 을 notification으로 보여준 뒤 다시 `getDrPlan`, `listDrRuns`, 탭별 list API를 호출해 최신 상태를 표시한다.

5. UI와 Backend 간 용어 매핑

UI 표현	API	Backend	DB/상태
DR Sites	<code>listDrSites</code> , <code>createDrSite</code> , <code>checkDrSite</code>	<code>DrSiteService</code>	<code>dr_site</code> , <code>dr_site_pair</code>
DR Plans	<code>listDrPlans</code> , <code>getDrPlan</code> , <code>createDrPlan</code>	<code>DrPlanService</code>	<code>dr_plan</code>
Action 버튼	<code>startDrSync</code> , <code>startDrFailover</code> , <code>startDrFailback</code> , <code>startDrReprotect</code> , <code>adoptDrReplica</code>	<code>DrRunService</code> , <code>DrRunExecutor</code>	<code>dr_run</code> , <code>dr_run_step</code> , <code>dr_event</code>
Overview 상태	<code>getDrPlan</code> , <code>listDrRuns</code>	<code>DrPlanService</code> , <code>DrRunService</code>	<code>dr_plan.state</code> , <code>dr_run.state</code>
Replica 탭	<code>listDrReplicas</code>	<code>DrProjectionService</code>	<code>dr_replica</code> , <code>dr_replica_disk</code>
Restore point 탭	<code>listDrRestorePoints</code>	<code>DrProjectionService</code>	<code>dr_restore_point</code> , <code>dr_restore_point_artifact</code>
Events 탭	<code>listDrEvents</code>	<code>DrProjectionService</code>	<code>dr_event</code>
Run progress	<code>listDrRunSteps</code>	<code>DrRunService</code>	<code>dr_run_step.progress</code>

6. 배포 전 리뷰 체크리스트

6.1 UI

- `drsite`, `drplan` section이 권한 기반으로 표시되는지 확인한다.
- DR Plans 목록에서 site 이름, direction, engine, RPO, action 버튼이 정상 표시되는지 확인한다.
- 상세 화면에서 Overview, Restore Points, Replica, Runs, Events 탭이 모두 열리는지 확인한다.
- 다크모드에서 `cross-dr` surface, border, secondary text 색상이 읽히는지 확인한다.
- action 실행 후 notification과 재조회가 동작하는지 확인한다.

6.2 Backend

- `DisasterRecoveryClusterServiceImpl.getCommands()` 에 신규 DR API command가 모두 등록되어 있는지 확인한다.
- `spring-disaster-recovery-context.xml` 에 DAO, service, orchestrator, adapter bean이 등록되어 있는지 확인한다.
- `DrPlanServiceImpl.getActionEligibility` 가 FTCTL, VMware Phase 1, V2K별 허용 action을 분리하는지 확인한다.
- `DrRunExecutorImpl` 이 성공/실패 모두에서 step/event/run 상태를 남기는지 확인한다.
- FTCTL adapter가 기존 `FtctlService` 성공 경로를 위임 호출하고 qemu/ftctl runtime을 Cloud DR에서 재구현하지 않는지 확인한다.
- V2K adapter가 기존 `import_vm_task` 를 추적하며 phase2 시작 경로를 직접 소유하지 않는지 확인한다.

6.3 배포

- Cloud backend는 변경 Maven module 산출물 기준으로 반영한다.
- UI는 `/usr/share/cloudstack-management/webapp` 의 `WEB-INF` 를 보존하고 정적 asset만 갱신한다.
- DB schema 적용 전 backup을 확보한다.
- 배포 후 `/client/` HTTP 200, management service status, DR API list smoke를 확인한다.
- 이번 구현 범위에서는 qemu/ftctl package 배포가 필수 항목이 아니다.